

❖ 已有几种查找方法:

顺序查找	$O(N)$
二分查找（静态查找）	$O(\log_2 N)$
二叉搜索树	$O(h)$ h 为二叉查找树的高度

$O(\log_2 N)$ 已经是相当不错的时间复杂度!

○

○

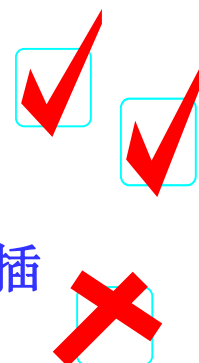
○

到底还有没有其他适应性广
而速度又快的查找方法呢?

[例5.1] 在登录QQ的时候，QQ服务器如何**核对你的身份**，以确定你就是该号码的主人？

【分析】 看看是否可以用**二分法**查找。

- 十亿 ($10^9 \approx 2^{30}$) 有效用户，用二分查找**30**次。
- 十亿 ($10^9 \approx 2^{30}$) $\times$ 1K $\approx$ 1024G，**1T**连续空间。
- 按有效QQ号大小有序存储：在连续存储空间中，**插入和删除一个新QQ号码**将需要**移动大量数据**。

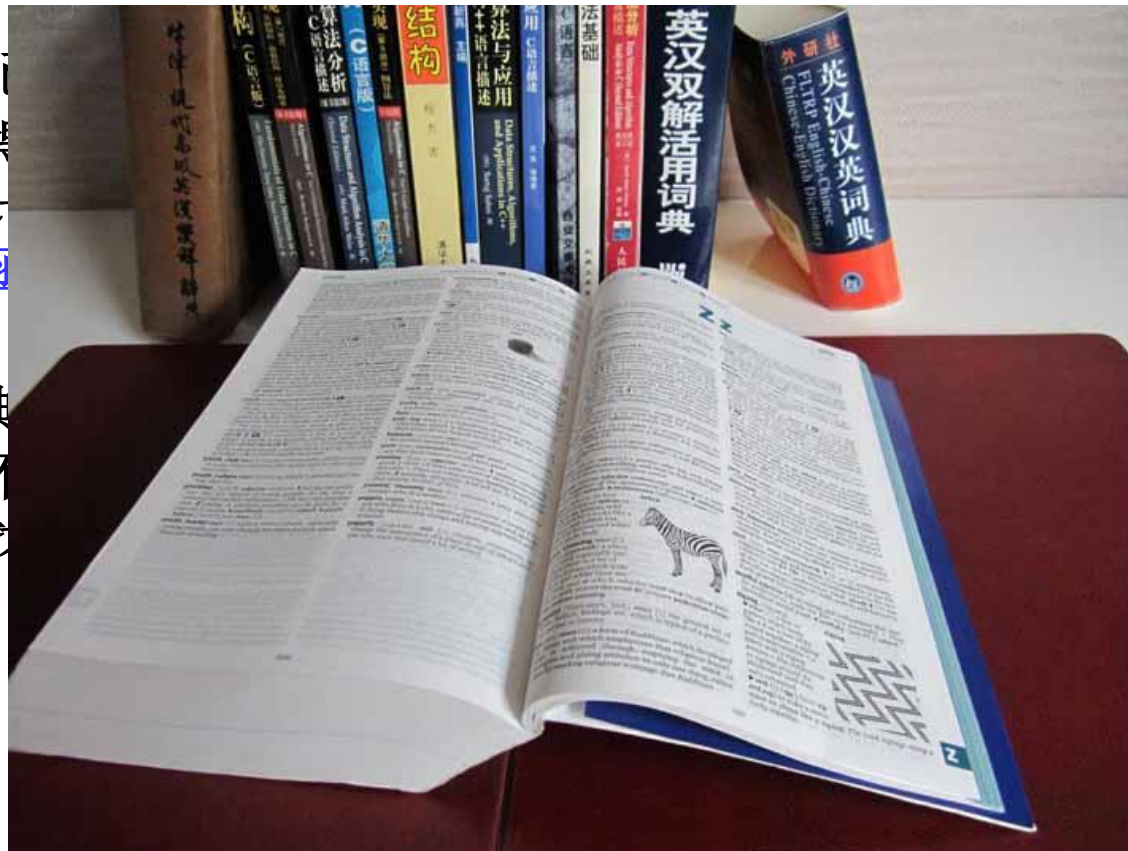


用不了二分查找，
我们该怎么办？

[例5.2] 查英文字典的过程——查询英文单词 “zoo”，你为什么不用二分法，而直接从字典的后面找？

► 我们
算”，得
这个“计
“散列函

► 查字典
（一般不
几种查找



“计
找到它。
中的

查找
时候）等

[例5.3] 网上搜索。搜索引擎是如何如此神速地把我们需要的有关信息找到的？

【分析】

关系。

Docs Terms	文档 ₁	文档 ₂		文档 _{m-1}	文档 _m
关键词 ₁	3: 1,12,20	0		2: 1, 22	3: 9,40,52
关键词 ₂	0	2: 11,22		4: 9,20,32,65	5: 5,9,10,32,35
.....					
关键词 _{n-1}	0	0		5: 3,9,10,32,56	10: 5,6,19,...,44
关键词 _n	5: 1,9,20,22,55	0		0	1: 7

如何在这个巨大无比的表格中查找特定的关键词？

【问题】如何能够在极短的时间内（比如1秒内）搜索到需要的关键词？

【答案】散列查找法是很好方法之一！

❖ 散列查找法的两项基本工作：

- 构造散列函数：确定关键词所在的存储位置的计算方法；
- 解决冲突：当多个关键词所在的存储位置相同时的解决方法。

❖ 并且，期望查找的时间复杂度好于 $O(\log_2 N)$ ，
——几乎是常量： $O(1)$ ，即查找时间与问题规模无关！

❖ 当然，散列查找法也有缺点和局限性.....

❖ 散列表的定义

形如“名字(Name)-属性(Attribute)”对的集合的“符号表(Symbol Table)”也叫做“散列表”(Hash Table, 即哈希表)。

类型名称:符号表 (SymbolTable)

数据对象集:符号表是“名字(Name)-属性(Attribute)”对的集合。

操作集:对于一个符号表 $\text{Table} \in \text{SymbolTable}$, 一个给定名字 $\text{Name} \in \text{NameType}$, 属性 $\text{Attr} \in \text{AttributeType}$, 以及正整数 TableSize , 符号表的基本操作主要有:

- 1、**SymbolTable InitializeTable(int TableSize):** 创建一个长度为 TableSize 的符号表;
- 2、**Boolean IsIn(SymbolTable Table, NameType Name):**
查找特定的名字 Name 是否在符号表 Table 中;
- 3、**AttributeType Find(SymbolTable Table, NameType Name):**
获取 Table 中指定名字 Name 对应的属性;
- 4、**SymbolTable Modify(SymbolTable Table, NameType Name, AttributeType Attr):**
将 Table 中指定名字 Name 的属性修改为 Attr ;
- 5、**SymbolTable Insert(SymbolTable Table, NameType Name, AttributeType Attr):**
向 Table 中插入一个新名字 Name 及其属性 Attr ;
- 6、**SymbolTable Delete(SymbolTable Table, NameType Name):**
从 Table 中删除一个名字 Name 及其属性。

- ❖ “散列 (Hashing)” 的基本思想是：以数据对象的关键字 **key** 为自变量，通过一个确定的 **函数关系 h** ，计算出对应的函数值 **$h(key)$** ，把这个值解释为数据对象的存储地址，并按此存放，即“**存储位置 = $h(key)$** ”。
- ❖ 散列方法中使用的计算函数称为“**散列函数**”（也称哈希函数），按这个思想构造的表称为“**散列表**”，所以它也是一种存储方法。
- ❖ 在查找某数据对象时，用同样的方法“**存储位置 = $h(key)$** ”计算出地址，将 **key** 与该地址单元中数据对象关键字进行比较，确定查找是否成功。
- ❖ 可能将 **不同的关键字** 映射到同一个散列地址上，即 **$h(key_i) = h(key_j)$** （当 **$key_i \neq key_j$** ），这种现象称为“**冲突(Collision)**”， **key_i** 和 **key_j** 称为“**同义词 (synonym)**”。
- ❖ 通常关键词的 **值域**（允许取值的范围）远远大于表空间的 **地址集**，所以说，**冲突不可能避免，只能尽可能减少**。

[例5.4] 有 $n = 11$ 个数据对象的集合，关键词是正整数，分别为18, 23, 11, 20, 2, 7, 27, 30, 42, 15, 34。如果符号表的大小用 $\text{TableSize} = 17$ （通常用一个素数），选取散列函数 h 如下：

$$h(\text{key}) = \text{key} \bmod \text{TableSize} \qquad \text{(公式 5.1)}$$

其中 mod 是求余运算，相当于C语言中的 $\%$ 运算。
用这个散列函数对11个数据对象建立查找表（忽略各关键词对应的属性部分，该例的关键词没有冲突）如下：

地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
关键词	34	18	2	20			23	7	42		27	11		30		15	

查找时，对给定关键词 key ，依然通过公式5.1计算出地址，再将 key 与该地址单元中关键词比较，若相等，则查找成功。

【定义】 设散列表空间大小为 m ，填入表中的元素个数是 n ，则称 $\alpha = n/m$ 为散列表的**装填因子 (Loading Factor)**。

比如查找： $\text{key} = 22$ ，地址是 $22 \% 17 = 5$ ，该地址空表示表中没有关键词22的信息。

装填因子 $\alpha = 11 / 17 \approx 0.65$ 。

实用时，常将散列表大小设计使得 $\alpha = 0.5 \sim 0.8$ 为宜。

比如查找： $\text{key} = 40$ ，地址是 $40 \% 17 = 6$ ，该地址存的是23， $40 \neq 23$ ，故还要采用后面介绍的**解决冲突**的策略才能确定。

[例5.5] 设计散列函数，使得当冲突或溢出不可避免时，如何处理使得表中没有空单元被浪费，同时插入、删除、查找等操作都能正确完成？

（注：原图中有“设计散列函数，使得冲突或溢出不可避免时，如何处理使得表中没有空单元被浪费，同时插入、删除、查找等操作都能正确完成？”以及“（注：原图中有“设计散列函数，使得冲突或溢出不可避免时，如何处理使得表中没有空单元被浪费，同时插入、删除、查找等操作都能正确完成？”）”等文字，已按上述内容整理。）

装填因子 $\alpha = ?$

10, 19

为了把字母 $a \sim z$ 映射到 $0 \sim 25$, 如何设计函数 $h(key) = ?$

$h(key) = key[0] - 'a'$

acos define float exp char
atan ceil floor clock ctime

	槽 0	槽 1
0	acos	atan
1		
2	char	ceil
3	define	
	exp	
5	float	floor
6		
.....		
25		

如果没有溢出，
 $T_{\text{查询}} = T_{\text{插入}} = T_{\text{删除}} = O(1)$

❖ 一个“好”的散列函数一般应考虑下列两个因素：

- 1. 计算简单，以便提高转换速度；
- 2. 关键词对应的地址空间分布均匀，以尽量减少冲突。

❖ 数字关键词的散列函数构造

1. 直接定址法

取关键词的某个线性函数值为散列地址，即

$$h(key) = a \times key + b \quad (a、b \text{ 为常数})$$

地址 $h(key)$	出生年份(key)	人数($attribute$)
0	1990	1285万
1	1991	1281万
2	1992	1280万
...		
10	2000	1250万
...		
21	2011	1180万

2. 除留余数法

散列函数为: $h(\text{key}) = \text{key} \bmod p$

例5.4: $h(\text{key}) = \text{key} \% 17$

地址 $h(\text{key})$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
关键词 key	34	18	2	20			23	7	42		27	11		30		15	

- 这里: $p = \text{TableSize} = 17$ 。也可以采用 $p \neq \text{TableSize}$;
- $\text{TableSize} = n/\alpha$; n --key集合的大小, α 装填因子上限;
- $p \leq \text{TableSize}$ 的某个最大素数。

TableSize	8	16	32	64	128	256	512	1024
p	7	13	31	61	127	251	503	1019

3. 数字分析法

- 取11位手机号码 key 的后4位作为地址：
- 散列函数为： $h(key) = atoi(key+7)$

- 若 key 集合大小 $n \leq 10000$ ，则比较合适。若取装填因子上限 $\alpha=0.8$ ，那么地址空间（表长） $TableSize = n/\alpha=12500$

- 若 key 集合大小 $n \leq 100$ ，则可以作如下二选一：
 - a、若取装填因子上限 $\alpha=0.8$ ，那么地址空间（表长） $TableSize = n/\alpha \approx 128$ ，取 $p=127$ ， $h(key) = atoi(key+7) \% p$
 - b、直接取后2位作为地址：
 $h(key) = atoi(key+9)$ ，而表长 $TableSize = n/\alpha \approx 127$ 。

3. 数字分析法

如果关键词 **key** 是18位的身份证号码：

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
3	3	0	1	0	6	1	9	9	0	1	0	0	8	0	4	1	9
省		市		区（县） 下属辖区 编号		（出生）年份				月份		日期		该辖区中的 序号		校验	

$$h_1(\text{key}) = (\text{key}[6] - '0') \times 10^4 + (\text{key}[10] - '0') \times 10^3 + (\text{key}[14] - '0') \times 10^2 + (\text{key}[16] - '0') \times 10 + (\text{key}[17] - '0')$$

$$h(\text{key}) = h_1(\text{key}) \times 10 + 10 \qquad \text{（当 key[18] = 'x' 时）}$$

$$\text{或 } h(\text{key}) = h_1(\text{key}) \times 10 + \text{key}[18] - '0' \qquad \text{（当 key[18] 为 '0' ~ '9' 时）}$$

❖ 字符关键词的散列函数构造

1. 一个简单的散列函数——ASCII码加和法

对字符型关键词key定义散列函数如下：

$$h(\text{key}) = (\sum \text{key}[i]) \bmod \text{TableSize}$$

2. 简单的改进——前缀字符移位法

改造如下：

$$h(\text{key}) = (\text{key}[0] + \text{key}[1] \times 2 + \text{key}[2] \times 4 + \dots + \text{key}[n-1] \times 2^{n-1}) \bmod \text{TableSize}$$

冲突严重：a3、b2、c1；
eat、tea；

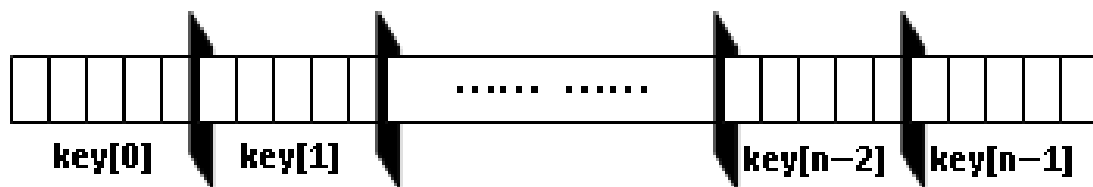
3. 好的散列函数——移位法

涉及关键词的所有n

仍然冲突：string、street、
strong、structure等等；
空间浪费： $3000/26^3 \approx 30\%$

$$h(\text{key}) = \left(\sum_{i=0}^{n-1} \text{key}[n-i-1] \times 32^i \right) \bmod \text{TableSize}$$

❖ 每位字符占5位二进制（即 $2^5 = 32$ ）。实现时不需要做乘法运算，只需一次左移5位来完成。这也是为什么选用32来代替27的原因。



```
Index Hash ( const char *Key, int TableSize )
{
    unsigned int h = 0;      /* 散列函数值，初始化为
0 */
    while ( *Key != '\0' ) /* 位移映射 */
        h = ( h << 5 ) + *Key++;
    return h % TableSize;
}
```

对32位字长的无符号整数，只有7个字符起作用： $32 \leq 7 \times 5$ 。不过在实际应用中已经不错了。